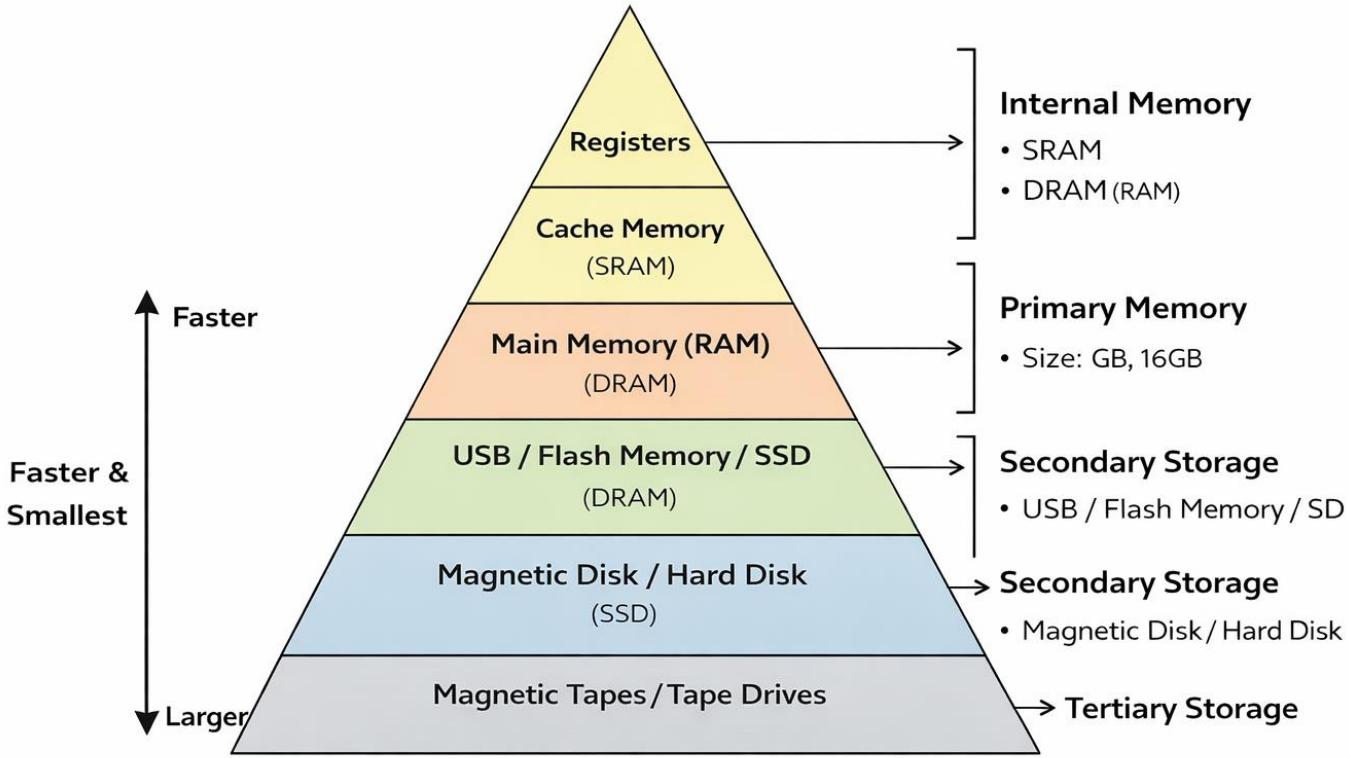
Unit-5

Vaibhav K Matalia

Memory Hierarchy



Memory Hierarchy

Memory	• Cache Memory • USB / Flash Memory	• SSD / USB/ • Magnetic Disk / SSSD

- Memory in a computer is arranged in **hierarchy (pyramid shape)**
- Pyramid shape shows:
  - **Size**
  - **Speed (access time)**
  - **Cost**
- Different levels of memory are used together in a system.
- This helps to **balance speed, cost, and storage capacity.**
- Top level → Small size, very fast, very costly
- Bottom level → Large size, slow, cheap
- As we go down:
  - **Size increases**
  - **Access time increases**
  - **Speed decreases**

- All levels are needed for efficient working of computer.

Register:-

- Smallest and fastest memory
- Located inside CPU
- Stores:
  - Data
  - Instructions
  - Addresses
- Size: 8-bit, 16-bit, 32-bit, 64-bit

- Types:
  - General purpose registers
  - Address registers
  - MBR (Memory Buffer Register)
- CPU directly communicates with registers.

Cache Memory:-

- Placed between CPU and RAM
- Faster than RAM
- Stores frequently used data
- Size: KB or MB
- Cache memory uses SRAM technology

Main Memory(RAM):-

- Also called Primary Memory
- Implemented using DRAM (Dynamic RAM)
- Size: GB (8GB, 16GB, etc.)
- It is volatile memory
- Data is lost when power is OFF
- Supports:
  - Multitasking
  - Multiprocessing

USB / Flash Memory / SSD

- Non-volatile storage
- Used for:
  - Data storage
  - File transfer
- SSD is faster than Hard Disk
- Access time is in microseconds
- Used in laptops and modern systems

Magnetic Disk (Hard Disk)

- Secondary storage device
- Stores operating system, software, files

- Size: GB or TB
- Slower than RAM and SSD
- Access time: milliseconds

Magnetic Tape / Tape Drive

- Largest storage capacity
- Size: TB or PB
- Used for Backup
- Not directly connected to CPU
- Very slow access time

Access Time Comparison:-

Register → ~ 1 nanosecond

Cache → ~ 10 nanoseconds

RAM → ~ 100 nanoseconds

SSD / Flash → microseconds

Hard Disk → ~ 10 milliseconds

Tape Drive → ~ 100 milliseconds

Cost and Speed

Registers → Most expensive, fastest

Cache → High cost, very fast

RAM → Medium cost

Hard Disk → Cheap

Tape → Cheapest, very slow

Note:-

1. Memory hierarchy improves Speed, Performance and Cost efficiency.
2. Frequently used data is kept in Registers and Cache.
3. Backup data is stored in Hard disk and Tape drive.

Questions

1.Consider a system with 2 level caches. Access time of L1 and L2 and Main memory are 1ns,10ns and 500ns.Hit ratio of L1 and L2 are 0.8 and 0.9.What is average access time of system ignoring search time within cache?

2.A system has two cache levels. L1 access time = 2 ns, L2 access time = 12 ns, Main memory access time = 200 ns. Hit ratio of L1 = 0.7 and L2 = 0.8. Find the average memory access time (AMAT).

Note:-Independent organization is used only when the problem explicitly says things like L1 and L2 are accessed in parallel, Independent memory modules, Parallel access to memories. Since nothing like that is mentioned here, we assume: Hierarchical organization (serial search).

Exercise:-

1. A processor uses two cache levels.
   L1 hit time = 2 ns
   L2 hit time = 12 ns
   Main memory time = 500 ns
   L1 hit ratio = 0.75
   L2 hit ratio = 0.8
   (a) Find AMAT
   (b) If L1 hit ratio increases to 0.9, find the new AMAT
   (c) Find percentage improvement in access time

2. A system has two cache levels connected in series.
L1 access time = 1 ns
L2 access time = 5 ns
Main memory time = 100 ns

Overall hit ratio of cache system = 0.95
Hit ratio of L1 = 0.8

Find the hit ratio of L2 and the average memory access time.

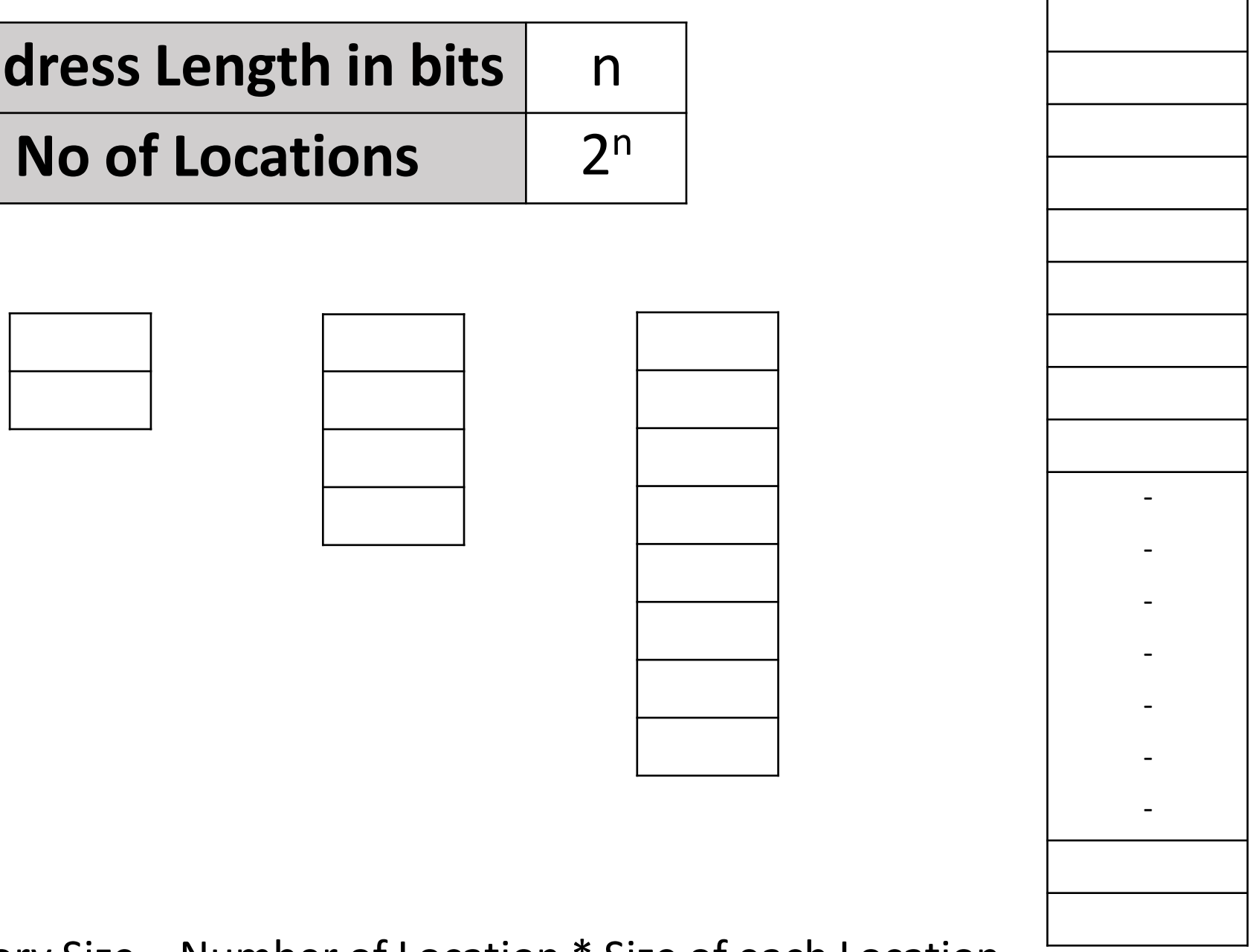
Units for Memory Management

10^3	1 Thousands
10^6	1 Million
10^9	1 Billion
10^{12}	1 Trillion

10^3	1 kilo
10^6	1 Mega
10^9	1 Giga
10^{12}	1 Tera
10^{15}	1 Peta

2^{10}	1 kilo
2^{20}	1 Mega
2^{30}	1 Giga
2^{40}	1 Tera
2^{50}	1 Peta

Address Length in bits	n
No of Locations	2^n



Memory Size = Number of Location * Size of each Location

- If a memory has a 10-bit address, then the total number of memory locations is:

$2^{10} = 1024$ locations.

- How do we determine the size of each memory location?
  - In 95% of cases, the size of each memory location is not directly given in the question.
  - If the question says “memory is byte addressable”, then:
    - ❖Each memory location = 1 Byte.
- If the question explicitly mentions the size of each memory location, then we must use that value.

Example:

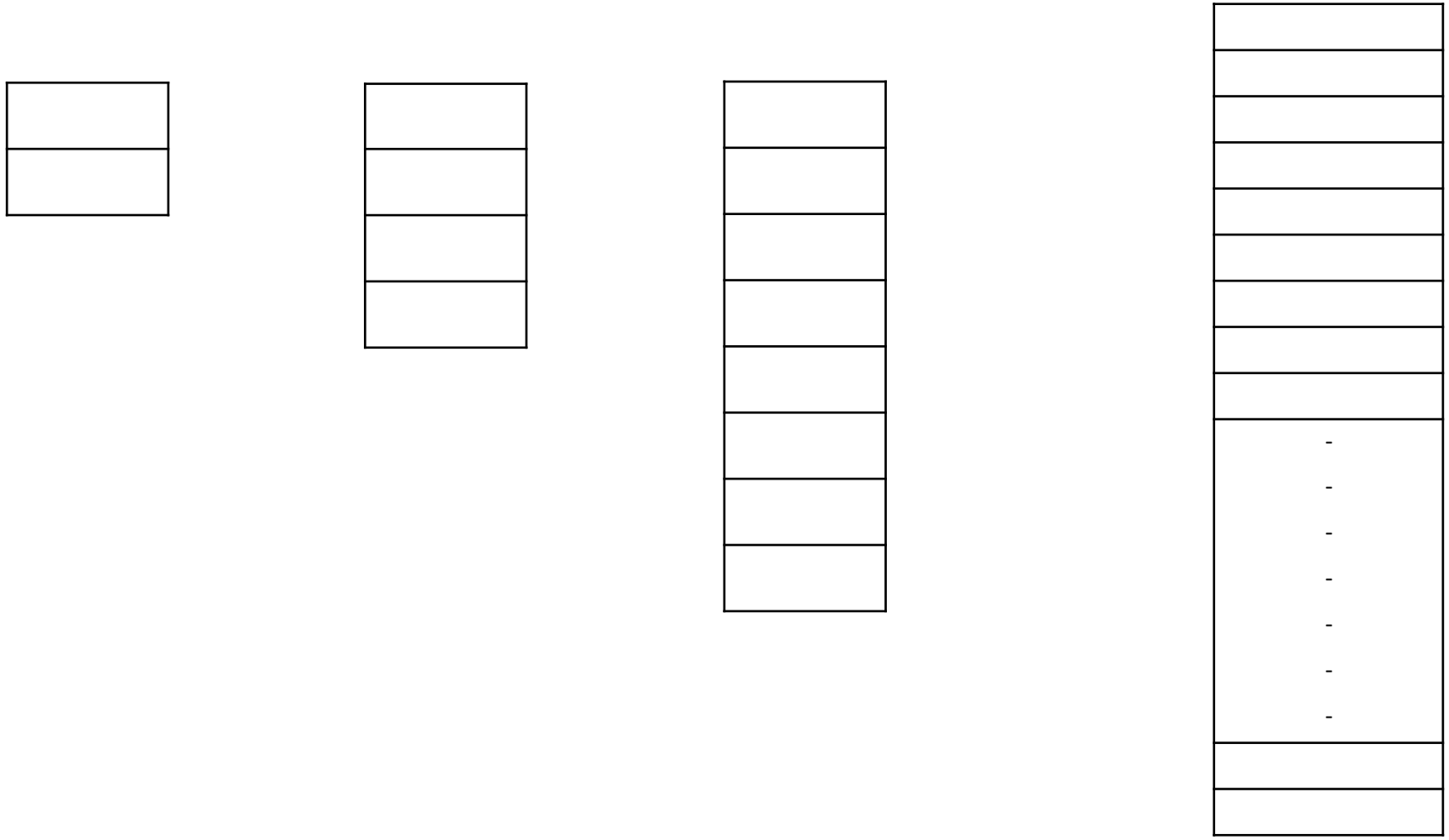
- “Each memory location is 2 bytes”
- “Each memory location is 3 bytes”

  → Use the given size for calculation.

Example:-Address size is 32-bits. Find the memory capacity.

Example:-CPU Generate 24-bit address. What is the memory size?

Address Length in bits	Upper Bound($Log_2 n$)
No of Locations	n



- If an address is n bits, then the total number of memory locations is:2^n locations.
- If the number of memory locations is N, then the address length is:$\log_2(N)$ bits.
- If the value of $\log_2(N)$ comes in a floating-point (decimal) number, we must take the upper bound (next higher integer).

Example 1:Memory has 10 locations. Find the length of address.

Example 2:Memory size = 512 KB.Find the address length.

Example 3:-A processor can support a maximum memory of 4 GB, where the memory is word-addressable (a word consists of two bytes). The size of the address bus of the processor is at __ least bits. (Gate-2016) (1 Marks)

Exercise Questions:-

1. A computer uses 36-bit addresses and memory is byte-addressable.
Find the maximum memory capacity.

2. A system has 20-bit address bus and memory is word-addressable.
If one word = 4 bytes, find the maximum memory size.

3.A memory has 2048 locations.
(a) Find the address length.
(b) If each location stores 2 bytes, find the total memory size.

4. A memory of size 64 KB is byte-addressable.Find the number of address bits required.

5. A system supports 1 GB memory and is word-addressable.If word size = 8 bytes, find the address bus size.

6. A memory chip has 16K × 8 organization.
(a) How many address lines are required?
(b) What is the total memory capacity?

7. A memory has 4096 locations and each location stores 16 bits. Find the memory size in KB.

8. A memory system is organized as 32K × 4.
Find:
(a) Number of locations
(b) Size of each location
(c) Total memory capacity

9. A memory has 50,000 locations. Each location stores 1 byte.
Find:
Minimum address length
Total unused address combinations

Cache Mapping And Its Types

- In computer organization, the CPU is directly connected to Cache memory, which is inside the CPU chip.
- There may be different levels of cache as well like Level 1,Level 2,Level 3.
- Data is first stored in hard disk / SSD.
- A small part of data is moved from:
  - ❖Hard disk → RAM
  - ❖RAM → Cache
  - ❖Cache → CPU registers
- Instead of using only one memory, we use multiple levels of memory.
- CPU works very fast (in nanoseconds). Cache memory also works in nanoseconds, so it matches CPU speed. Hard disk works in milliseconds.There is a big speed gap between CPU and hard disk. To reduce this gap, we use cache and RAM between CPU and hard disk.

- Main memory (RAM) is divided into blocks. Each block contains words.
- A word is the minimum addressable unit of memory.
- Cache is divided into lines.
- Data is transferred from RAM block → Cache line.
- When a block is brought from RAM to cache:
  - ❖In which **cache line** should it be placed?
  - ❖How will **CPU find that data** in cache?
- The method that decides:
  - ❖Where to place a memory block in cache
  - ❖How CPU will access it
    is called **Cache Mapping**.

- So, Cache mapping decides which block of main memory goes into which cache line and how the CPU finds it.
- There are three types of cache mapping:

1. Direct Mapping
2. Fully Associative Mapping
3. k-way Set Associative Mapping

Direct Mapping

Cache Memory

L0	B0	B4	B8	B12
L1	B1	B5	B9	B13
L2	B2	B6	B10	B14
L3	B3	B7	B11	B15

Cache Size = 16 words

Block size = 4 words

Main Memory

W0	W1	W2	W3	B0
W4	W5	W6	W7	B1
W8	W9	W10	W11	B2
W12	W13	W14	W15	B3
W16	W17	W18	W19	B4
...	...	...	...	
W124	W125	W126	W127	B31

Main Memory = 128 words

Block size = 4 words → 32 blocks

- Assume the size of main memory = 128 words.
- Memory size can be represented in:
  - ❖Bytes (KB, MB, GB), or
  - ❖Words (used more in computer architecture).
- Main memory is divided into blocks.Each block contains multiple words.
- Cache memory is divided into lines.
  Block size = Line size
  Total number of blocks=Size of main memory/Number of words per block
  Total number of cache lines= Size of cache memory / Number of words per line
- Main memory has 128 words, but cache has only 16 words.All words cannot be stored in cache at the same time.

- So, we must decide:
  - ❖Which memory block will come into cache?
  - ❖In which cache line it will be stored?
- In direct mapping, we use the formula:
  Line number = Block number mod Number of lines (K mod N)
  K = Block number and N = Number of cache lines
- Mapping is fixed:
  - ❖Block 0 → Line 0
  - ❖Block 1 → Line 1
  - ❖Block 2 → Line 2
  - ❖and so on…
- At one time, only one block can be stored in one cache line.

- Main memory size = 128 words → addresses from 0 to 127.To represent 128 addresses, we need 7 bits.So, physical address size = 7 bits
- The address is divided into:
  - ❖Block number
  - ❖Block offset (word number inside the block)
- Block number is divided into:
  - ❖Tag
  - ❖Line number
- Line 0 can store blocks like:B0, B4, B8, B12, …, B28.Which block is currently in Line 0 is identified using the Tag.

- If the block required by CPU is already present in cache is called a Cache Hit.If Block 5 is in cache and CPU requests a word from Block 5 → Cache Hit
- If the block is not present in cache is called a Cache Miss.If Block 9 is required but Block 5 is currently in that line → Cache Miss
- In case of cache miss:
  - ❖The old block is removed.
  - ❖The new block is loaded into cache.
  - ❖This process is called Cache Replacement.

Q Consider a computer system with a byte-addressable primary memory of size 232 bytes. Assume the computer system has a direct-mapped cache of size 32 KB (1 KB = 210 bytes), and each cache block is of size 64 bytes. The size of the tag field is _____ bits. (GATE 2021)

(a) 17 (b) 18 (c) 15 (d) 9

Q Consider a machine with byte addressable memory of 232 bytes divided into blocks of size 32 bytes. Assume a direct mapped cache having 512 cache lines is used with this machine. The size of tag field in bits is ___.(Gate-2017) (2 Marks)

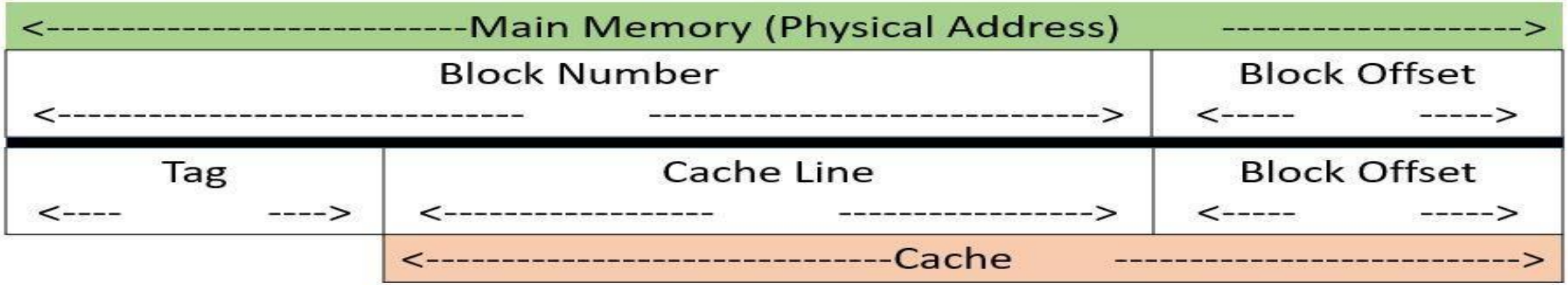
(A) 12 (B) 16 (C) 18 (D) 24

Q Consider a direct mapped cache of size 32 KB with block size 32 bytes. The CPU generates 32 bit addresses. The number of bits needed for cache indexing and the number of tag bits are respectively (Gate-2005) (2 Marks)

(A) 10, 17 (B) 10, 22 (C) 15, 17 (D) 5, 17

Q Consider a machine with a byte addressable main memory of 2^{20} bytes, block size of 16 bytes and a direct mapped cache having 2^{12} cache lines. Let the addresses of two consecutive bytes in main memory be $(E201F)_{16}$ and $(E2020)_{16}$. What are the tag and cache line address (in hex) for main memory address $(E201F)_{16}$? (Gate-2015) (1 Marks)

(A) E, 201 (B) F, 201 (C) E, E20 (D) 2, 01F



Q. For a direct-mapped cache, 4 bits are used for the tag field and 12 bits are used to index into a cache block. The size of each cache block is one byte. Assume that there is no other information stored for each cache block.

Which ONE of the following is the CORRECT option for the sizes of the main memory and the cache memory in this system (byte addressable), respectively? (Gate 2025)

64 KB and 4 KB

128 KB and 16 KB

64 KB and 8 KB

128 KB and 6 KB

Q. Consider a memory system with 1M bytes of main memory and 16 K bytes of cache memory. Assume that the processor generates 20-bit memory address, and the cache block size is 16 bytes. If the cache uses direct mapping, how many bits will be required to store all the tag values?

[Assume memory is byte addressable, $1K=2^{10}$, $1M=2^{20}$] (Gate 2025)

A) 6×2^{10}

B) 8×2^{10}

C) 2^{12}

D) 2^{14}

Fully Associative Mapping

Main Memory and Block Formation

- The size of main memory is 128 words.
- A word is considered the addressable unit of memory (it can also be considered as bytes in some cases).
- Each block contains 4 words.
- Therefore, Number of blocks in main memory = 128 / 4 = 32 blocks.

Cache Organization

- Cache size is 16 words.
- Size of one cache line is equal to the block size, i.e., 4 words per cache line.
- Hence,Number of cache lines = 16 / 4 = 4 cache lines.

CL-0
CL-1
CL-2
CL-3

B-0
B-1
B-2
B-3
B-4
B-5
B-6
B-7
B-8
B-9
B-10
B-11
B-12
B-13
B-14
B-15

Mapping Concept

- In fully associative mapping, any block of main memory can be placed in any cache line.
- There is no fixed relationship:
  - Block 0 can go to any cache line.
  - Block 1 can go to any cache line.
  - Block 2 can go to any cache line, and so on.
- Unlike direct mapping, no cache line is reserved for a specific block.

Physical Address Format

- Main memory size = 128 words, so addresses range from 0 to 127.
- To represent 128 locations, we need 7 bits:2^7=128

- Physical address is divided into:
  1. Block number (Tag)
  2. Block offset

Block Offset Calculation

- Each block contains 4 words.
- To select one word out of 4 words, we need:$\log_2(4)=2$ bits.
  Therefore,Block offset = 2 bits

Tag Field

- Since any block can be stored in any cache line, there is no cache line index in the address.
- The block number itself is used as the Tag.

- Thus, physical address format is:Tag (5 bits)|Block Offset (2 bits)

Note:-5 bits for block number because 32 blocks = 2^5

Example:-Suppose the tag stored in cache line 0 is:00100

- Block offset is of 2 bits.
- Possible physical addresses of Block 4 are:

0010000

0010001

0010010

0010011

(Offset values: 00, 01, 10, 11)

- This means Block 4 is currently stored in cache line 0.
- If the CPU requests any of these addresses, it will be a cache hit.

Advantages:-

- Cache space is used more efficiently because any block can be placed anywhere.
- Cache lines do not remain unused unnecessarily.
- This increases the cache hit ratio compared to direct mapping.

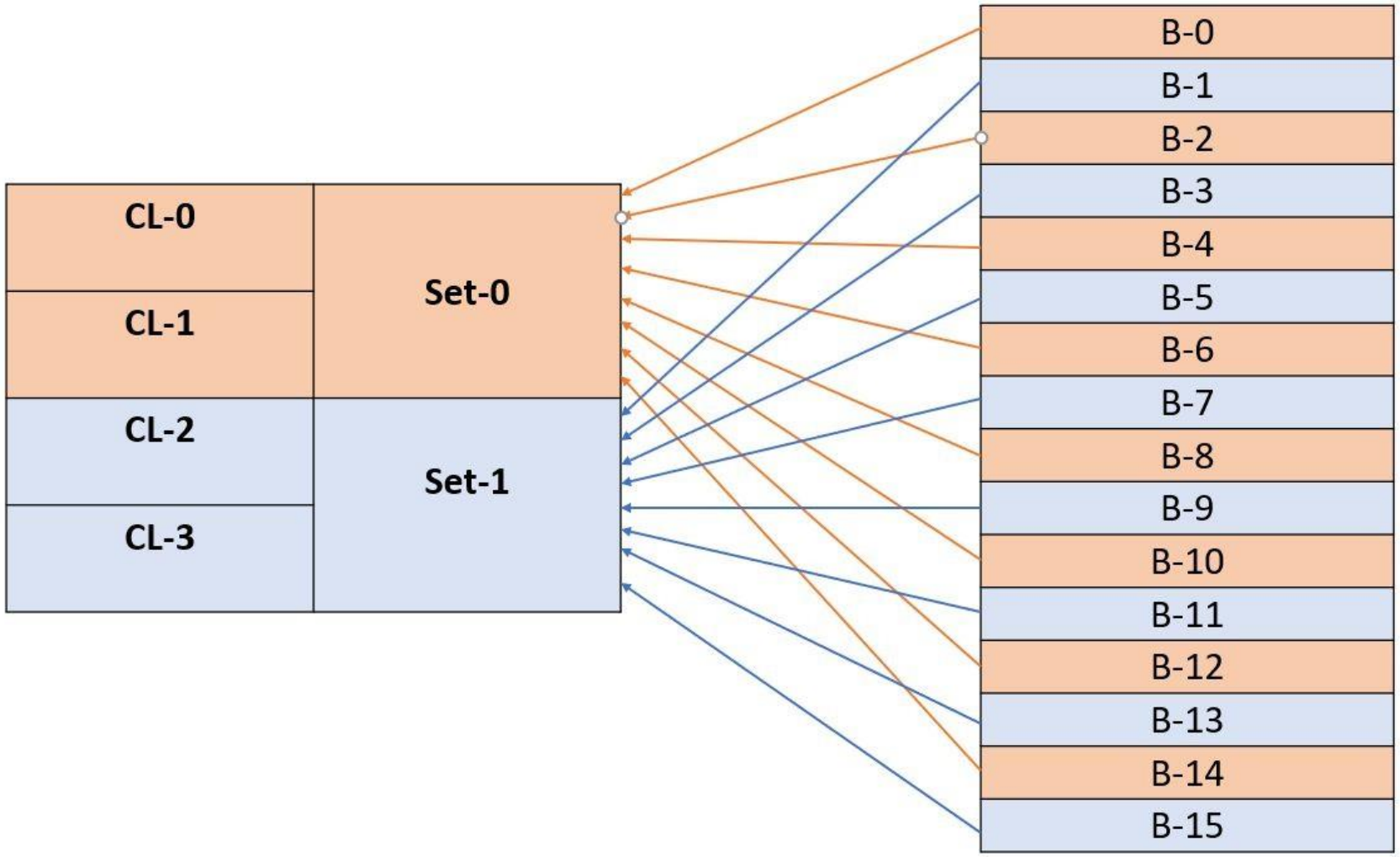
Disadvantage

- Since a block can be in any cache line, the CPU must Compare the requested tag with all cache line tags simultaneously.
- This requires multiple comparators, making the hardware more complex, more expensive and slightly slower.

Question:-Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find out the: Number of bits in tag and Tag directory size?

K-way set associative mapping

- K-way set associative mapping is a combination of direct mapping and fully associative mapping.
- In set associative mapping, a value K is given, such as 2-way set associative, 4-way set associative, etc.Here, K represents the number of cache lines in each set.
- The cache is divided into several sets.
- The total number of sets is calculated as:
  Number of sets=Number of cache lines/*K*
- To map a block from main memory into cache, the set number is first determined using the concept of direct mapping.
- The formula used to find the set number is:
  Set number=(Block number)mod(Number of sets)


CL-0
CL-1
CL-2
CL-3
Set-0
Set-1
B-0
B-1
B-2
B-3
B-4
B-5
B-6
B-7
B-8
B-9
B-10
B-11
B-12
B-13
B-14
B-15

- For example, if the cache is 2-way set associative and the number of sets is 2, then for block number 0:-0mod2=0. Therefore, block 0 will be mapped to set 0.
- Inside the selected set, the block can be placed in any one of the cache lines of that set.This follows the associative mapping concept.Hence, this mapping technique is a mixture of direct mapping (for set selection) and associative mapping (within the set).
- The physical address generated by the CPU is divided into:
    1. Block number
    2. Block offset
- The block number is further divided into:
    1. Tag
    2. Set number
    3. Block offset

- The tag indicates which block is currently stored in the selected set.
- The size of the tag determines how many different blocks can be associated with the same set.
- For example, if the tag size is 4 bits, then:2^4=16.This means 16 different blocks can be mapped to the same set.

Example:- Suppose the CPU generates the address:0010000

- It can be divided as:
  0010 = Tag
  0 = Set number
  00 = Block offset

- The CPU compares the tag part of the address with the tags stored in the cache lines of the selected set.
- If the selected set is set 0, and it has two cache lines (line 0 and line 1), then the tag is compared with the tags of both line 0 and line 1.
- If any tag matches, it is a cache hit.If no tag matches, it is a cache miss.

Question 1:-Consider the main memory size is of 128 KB, the cache size is of 16 KB, the block size is of 256 B, the set size is 2. Find Tag.

Question 2:-Main Memory = 32 GB, Cache Size = 32 KB, Block Size = 1 KB and it is a 4-way set associative cache?

Question 3:-Consider a set-associative cache of size 2KB (1KB=2^10 bytes) with cache block size of 64 bytes. Assume that the cache is byte-addressable and a 32 -bit address is used for accessing the cache. If the width of the tag field is 22 bits, the associativity of the cache is __.(GATE 2021) (1 MARKS)

Question The size of the physical address space of a processor is 2^P bytes. The word length is 2^W bytes. The capacity of cache memory is 2^N bytes. The size of each cache block is 2^M words. For a K-way set-associative cache memory, the length (in number of bits) of the tag field is (Gate-2018) (2 Marks)

Question:-A computer has a 256 KByte, 4-way set associative, write back data cache with block size of 32 Bytes. The processor sends 32-bit addresses to the cache controller. Each cache tag directory entry contains, in addition to address tag, 2 valid bits, 1 modified bit and 1 replacement bit. The number of bits in the tag field of an address is (Gate-2012) (2 Marks)
(A) 11 (B) 14 (C) 16 (D) 27

Question:-A cache memory unit with capacity of N words and block size of B words is to be designed. If it is designed as direct mapped cache, the length of the TAG field is 10 bits. If the cache unit is now designed as a 16-way set-associative cache, the length of the TAG field is __ bits.

Question:-The width of the physical address on a machine is 40 bits. The width of the tag field in a 512 KB 8-way set associative cache is __ bits.

Question:- A 4-way set-associative cache memory unit with a capacity of 16 KB is built using a block size of 8 words. The word length is 32 bits. The size of the physical address space is 4 GB. The number of bits for the TAG field is

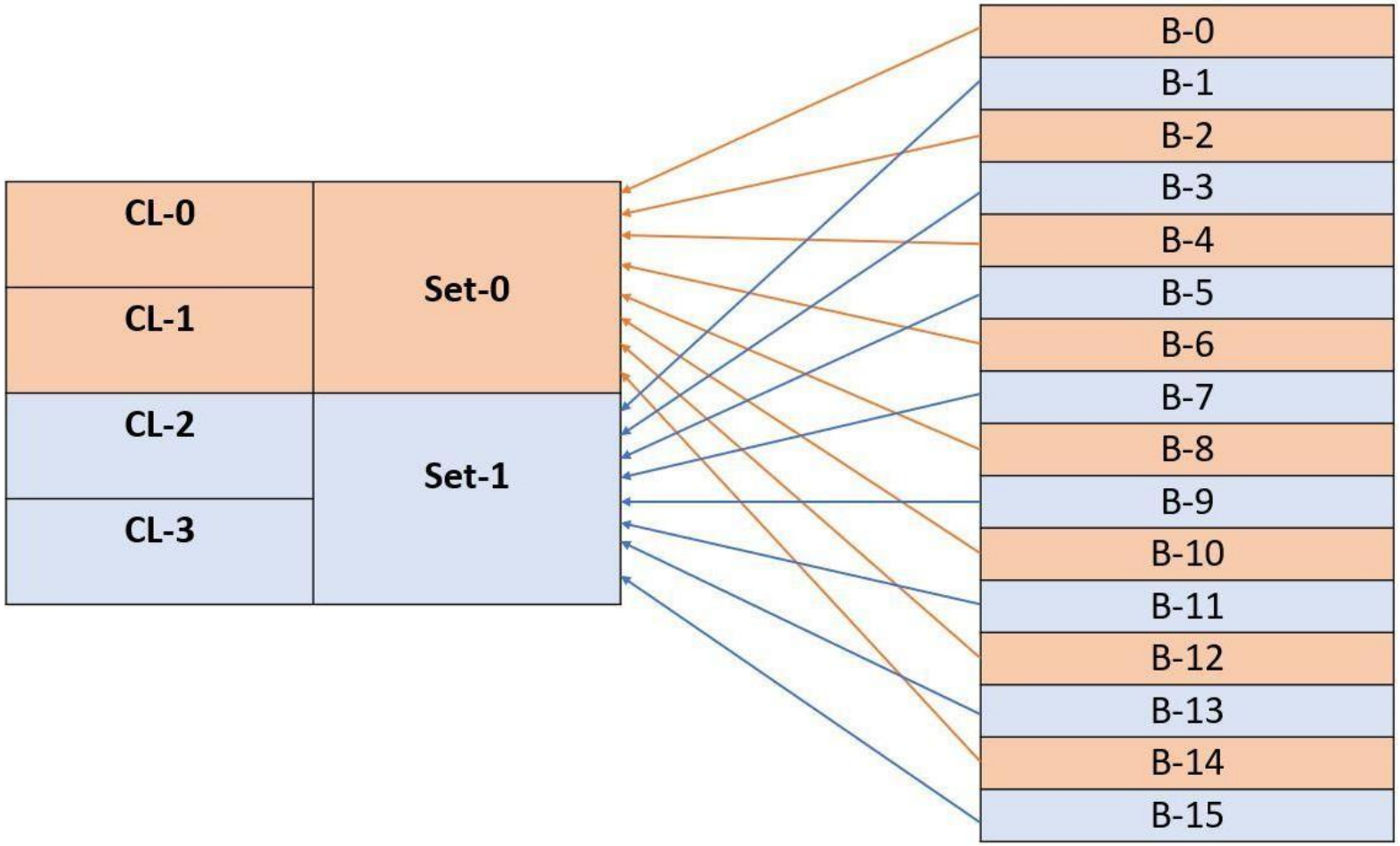
Q In a k-way set associative cache, the cache is divided into v sets, each of which consists of k lines. The lines of a set are placed in sequence one after another. The lines in set s are sequenced before the lines in set (s+1). The main memory blocks are numbered 0 onwards. The main memory block numbered j must be mapped to any one of the cache lines from. **(Gate-2013) (1 Marks)**

(A) (j mod v) * k to (j mod v) * k + (k-1)

(B) (j mod v) to (j mod v) + (k-1)

(C) (j mod k) to (j mod k) + (v-1)

(D) (j mod k) * v to (j mod k) * v + (v-1)



Locality of Reference

- Locality of reference means that the processor tends to access the same memory locations repeatedly.
- In other words, if the CPU accesses a particular memory block once, there is a high chance that it will access the same block again in the near future.
- It is not possible to store all the data of main memory in cache memory because cache memory has a very small size compared to main memory.
- So, data is first brought from main memory to cache memory, and then the CPU accesses the data from cache.
- The question is: Which block should be brought from main memory to cache memory, and how should it be selected?
- For this, we use the concept of **locality of reference**.

Spatial Locality

- Spatial locality means “space”.If a word is accessed now, then the words located next to it (adjacent words) are likely to be accessed next.
- So, when the CPU accesses one word, the entire block containing that word is brought into cache memory.
- Example: If the CPU needs the second word from block number 0, the whole block is loaded into cache. This is because we assume that after the second word, the CPU may request the third word or the first word.
- If the CPU later requests the first or third word, it will be found in cache memory, resulting in a **cache hit**.
- If a block contains more words, the chance of cache hit increases due to spatial locality.

Temporal Locality

- Temporal locality means “time.”
- If a word is accessed now, then the same word is likely to be accessed again in the near future.

Question:-The principle of locality justifies the use of: (Gate-1995) (1 Marks)

a) Interrupts

b) DMA

c) Polling

d) Cache Memory

Question:- More than one word are put in one cache block to (Gate-2001) (1 Marks)

(A) Exploit the temporal locality of reference in a program

(B) Exploit the spatial locality of reference in a program

(C) Reduce the miss penalty

(D) None of the above

Cache Replacement Policies

- We do not replace the cache memory itself; we replace the **data stored inside the cache**.
- All the blocks of main memory cannot be stored in cache memory.
  Assume that the cache is already full and the CPU requests a new block, for example **Block 5**.
- Block 5 contains the word that the CPU needs (or the CPU has generated the address for a word in Block 5).
- In this case, one of the existing blocks in the cache must be removed and replaced with Block 5.
- For example, out of Block 1, Block 2, and Block 3, one block will be sent back to main memory, and Block 5 will be loaded into cache memory.

- There are several methods to decide which block should be replaced. These are called **cache replacement policies**:
  1. FIFO (First In First Out)
  2. LRU (Least Recently Used)
  3. MRU (Most Recently Used)
  4. Random
- The main objectives of cache replacement policies are:
  - To reduce miss penalty
  - To increase cache hit ratio
- In **direct mapping**, cache replacement policies are not used because each main memory block has a fixed cache line where it must be placed.

- Cache replacement policies are mainly used in:
  - Set associative mapping
  - Fully associative mapping
- This is because, in these two methods, a block from main memory can be placed in any cache line of the cache.

Least Recently Used:-

The block that has not been used for the longest period of time in the past is replaced first.

Example:-Consider a fully associative cache with 8 cache lines (0 to 7) and the following sequence of memory block requests:4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

If the LRU replacement policy is used, which cache line will contain memory block 7?

- In the above example, when block **45** is requested, the cache is already full. So, the LRU policy is applied.
- To place block 45 in cache, we check which block in the cache has not been used for the longest time in the past.
- That block is removed, and block 45 is loaded into the cache.

Example:-Let the block in the sequence:7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 and the cache memory has 4 lines.

Example:-Consider 4 way set associative cache with total 16 cache lines.Main memory block requests are:0,255,1,4,3,8,133,159,216,129,63,8,48,32,73,92,155

Exercise 1:-Consider a fully associative cache with 5 cache lines.
The block reference string is:1, 2, 3, 4, 5, 1, 6, 2, 7, 3, 8, 2, 1, 9, 4

Using LRU replacement policy:

1. Find the number of cache hits
2. Find the number of cache misses
3. What are the final contents of cache?

Exercise 2:-The cache has 4 lines and uses LRU replacement policy.
Block reference string:0, 1, 2, 3, 0, 4, 1, 5, 2, 0, 1, 6

Find:

- Total cache hits
- Total cache misses
- Final cache state

Exercise 3:-A cache memory has 3 cache lines and follows LRU replacement policy.
Block reference string:2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Find:

- Hit ratio
- Miss ratio
- Final cache contents

FIFO (First In First Out) Replacement Policy

- FIFO stands for **First In First Out**.
- The block that enters the cache memory first will be replaced first when a new block needs to be loaded.
- In other words, the **oldest block in the cache** is removed first.

Example:-Consider a 4-way set associative cache with a total of 16 cache lines. The sequence of main memory block requests is:0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155

- In this example, all blocks are placed into their respective sets using the formula: Set number = Block number mod N (where N is the number of sets)

- This process continues until block 48 arrives and the cache becomes full. After that, the FIFO replacement policy is applied.

Example:-Let the block request sequence be:7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.Assume the cache memory has 4 cache lines.

- Using the FIFO replacement policy, the block that entered the cache first will be replaced first whenever the cache becomes full and a new block is requested.

Exercise:-A 2-way set associative cache has 8 cache lines.
Block reference sequence:0, 4, 8, 12, 16, 0, 4, 20, 24, 8, 12, 28
Using FIFO replacement policy, find:

Total cache hits

Total cache misses

Final contents of Set 0

Random:-

- The cache does not check which block is oldest OR which block was used recently. It just selects one block randomly and removes it.
- This policy does not need access information.
- It was used in some early processors such as ARM architectures.
- In modern systems, it is rarely used because better policies (like LRU) give higher performance.

Most Recently Used:-

- Cache keeps track of which block was accessed last.
- When replacement is needed the most recently accessed block is evicted.
- Idea: If a block was just used, it may not be needed again immediately.

- MRU works well when the program accesses memory in a cyclic pattern.

What is cyclic patterns:-

- The same set of blocks is accessed repeatedly in a fixed order (loop).

Example block request sequence:1, 2, 3, 4, 5, 1, 2, 3, 4, 1, 2

- Here Blocks 1,2,3 are accessed again and again in a cycle. So this is a cyclic (looping) access pattern.

Example:-

Assume:Cache size = 4 blocks and

Block request sequence =1, 2, 3, 4, 5, 1, 2, 3, 4, 1, 2

Exercise 1:-Assume:Cache size = 3 blocks
Block request sequence =1, 2, 3, 4, 2, 5, 2, 6

Using MRU (Most Recently Used) replacement policy, find:

Number of cache hits,Number of cache misses and Final cache contents.

Exercise 2:-Cache size = 4 blocks

Block reference string =2, 3, 4, 5, 3, 6, 7, 3, 8, 3

Using MRU replacement policy, determine:Total cache hits,Total cache misses and Final contents of cache

Read Policy

- Read policy explains what the cache does when CPU wants to READ data.

1.Read Hit:-A read hit happens when the required data is already present in cache.

Steps:

1. CPU sends address to cache
2. Cache checks its tags
3. Data is found in cache
4. Data is sent directly to CPU

Example:

Cache contains blocks:{10, 20, 30}

CPU wants block 20.Since 20 is already in cache.

- Cache gives data directly
- Main memory is not accessed
- Fast access

2.Read Miss

- A read miss happens when the required data is NOT in cache.

Steps:

1. CPU requests a block
2. Cache checks → not found
3. Cache fetches block from main memory
4. Block is loaded into cache. Data is sent to CPU(If cache is full, replacement policy is used)

Example:

- Cache contains blocks:{10, 20, 30}
- CPU wants block 40.Since 40 is not in cache: Read Miss
- Cache fetches 40 from RAM
- Stores 40 in cache
- Sends 40 to CPU
- If cache size is 3 blocks, one block (say 10) is removed using FIFO/LRU etc.

Load / Fetch Policy

- Load or fetch policy decides: "How much data should be brought into cache when a miss occurs?”
- There are two main policies:

1.Demand Fetch:-

- Only the required block is loaded into cache.
- When CPU requests a block:
  - ✓Load only that block
  - ✓Do not load extra blocks

Example:

- CPU requests block 100
- Cache loads only:100.It does NOT load 101, 102, 103.This is called demand fetch.

Advantage:

- Less cache space wasted

Disadvantage:

- If program needs nearby blocks, more misses occur.

2.Prefetch

- Cache loads the requested block and also some nearby blocks in advance.This is based on locality of reference.
- When CPU requests block 200:
  - ✓Cache loads 200
  - ✓Also loads 201, 202 (next blocks)

Advantage:

- Reduces future cache misses
- Improves performance

Disadvantage:

- May load unnecessary blocks
- Wastes cache space if prediction is wrong

Cache Write Policy

- Till now, we studied cache read operation.
- In read operation:
  - ✓CPU requests a word or block.
  - ✓If the block is not present in cache, it is read from main memory and stored in cache.
  - ✓CPU can generate read requests to both cache and main memory.
- But CPU does not only read data — it also writes (updates) data.
- When CPU updates data in cache, the original data in main memory must also be updated. This process is called Write Propagation.
- There are two methods of cache write:
  - ❑Write Through
  - ❑Write Back

1.Write Through

- In write-through method:Data is written to cache and main memory at the same time.

Advantages:

- Data consistency(cache and main memory will always have same value for content)

Disadvantages:-

- Time consuming because for write operation main memory is accessed irrespective of hit or miss.

Note: In read operation, we have cache hit and cache miss but In write operation, even if cache hit occurs, CPU still access main memory.

Average Access Time (Write Through)

1. Simultaneous Memory Access:

$$T_{avg(read)} = H \times T_{cache} + (1 - H) \times T_{main}$$

2. Hierarchical Memory Access:

$$T_{avg(read)} = H \times T_{cache} + (1 - H) \times (T_{cache} + T_{main})$$

3. Write Access:

$$T_{avg(write)} = \max(T_{cache}, T_{main}) = T_{main}$$

Note:-If Cache time = 10ns and Main memory time = 90ns then $T_{avg(write)}$=90ns

Overall Average Access Time

$$T_{avg} = (Fraction\ of\ Read) \times T_{avg(read)} + (Fraction\ of\ Write) \times T_{avg(write)}$$

Question:-A system has a write through cache with access time of 100ns and hit ratio of 90%.The main memory access time is 500ns.The 70% of memory references are for read operations.Find:

1.Average memory access time for read operations only

2.Average memory access time for write operations only

3.Average memory access time for read-write operations only

2. Write Back

In write-back method:

- ❑CPU updates only the cache.
- ❑Main memory is updated only when the block is replaced (evicted) from cache.

Advantage:-

Time Saving

Disadvantage:-

Data inconsistency

Average Access Time (Write Back)

1. Simultaneous Memory Access:

$$T_{avg(read/write)} = H \times T_{cache} + (1 - H) \times \left(T_{main} + T_{write_back}\right)$$

2.Hierarchical Memory Access:

$$T_{avg(read/write)} = H \times T_{cache} + (1 - H) \times \left(T_{cache} + T_{main} + T_{write_back}\right)$$

3.Write Allocate and No-Write Allocate

Write Allocate: On write miss, block is first copied to cache. Then CPU writes to cache.

No-Write Allocate: On write miss, block is not copied to cache. CPU writes directly to main memory.

For better performance:-

Write through uses no-write allocate.

Write back uses write allocate.

Types of Miss

Compulsory Miss:-When CPU demands for any block for the first time then definitely a miss is going to occur as the block needs to be brought into the cache, it is known as Compulsory miss.

Capacity Miss:-Occur because blocks are being discarded from cache because cache cannot contain all blocks needed for program execution.

Conflict Miss:-In the case of set associative or direct mapped block placement strategies, conflict misses occur when several blocks are mapped to the same set or block frame; also called collision misses or interference misses.

Q Consider a 2-way set associative cache with 256 blocks and uses LRU replacement, Initially the cache is empty. Conflict misses are those misses which occur due the contention of multiple blocks for the same cache set.
Compulsory misses occur due to first time access to the block. The following sequence of accesses to memory blocks (0, 128, 256, 128, 0, 128, 256, 128, 1, 129, 257, 129, 1, 129, 257, 129) is repeated 10 times. The number of conflict misses experienced by the cache is ____. (Gate-2017) (2 Marks)

Q Consider a Direct Mapped Cache with 8 cache blocks (numbered 0-7). If the memory block requests are in the following order 3, 5, 2, 8, 0, 63, 9,16, 20, 17, 25, 18, 30, 24, 2, 63, 5, 82,17, 24. Which of the following memory blocks will not be in the cache at the end of the sequence? (GATE-2007) (2 Marks)
(A) 3 (B) 18 (C) 20 (D) 30

Q Consider a small two-way set-associative cache memory, consisting of four blocks. For choosing the block to be replaced, use the least recently used (LRU) scheme. The number of cache misses for the following sequence of block addresses is 8, 12, 0, 12, 8 (Gate-2004) (2 Marks)

(A) 2 (B) 3 (C) 4 (D) 5